Automating Workflows (make)

Lecture 15

Dr. Colin Rundel

From scripts to a workflow

By the end of this lecture, you should be able to:

- describe a project as files and dependencies;
- write rules that build its outputs in the right order;
- predict what needs to rerun after an input changes;
- check that a build works from scratch and skips completed work.

We will use one Python, R, and Quarto project throughout.

Automating a project

A typical project

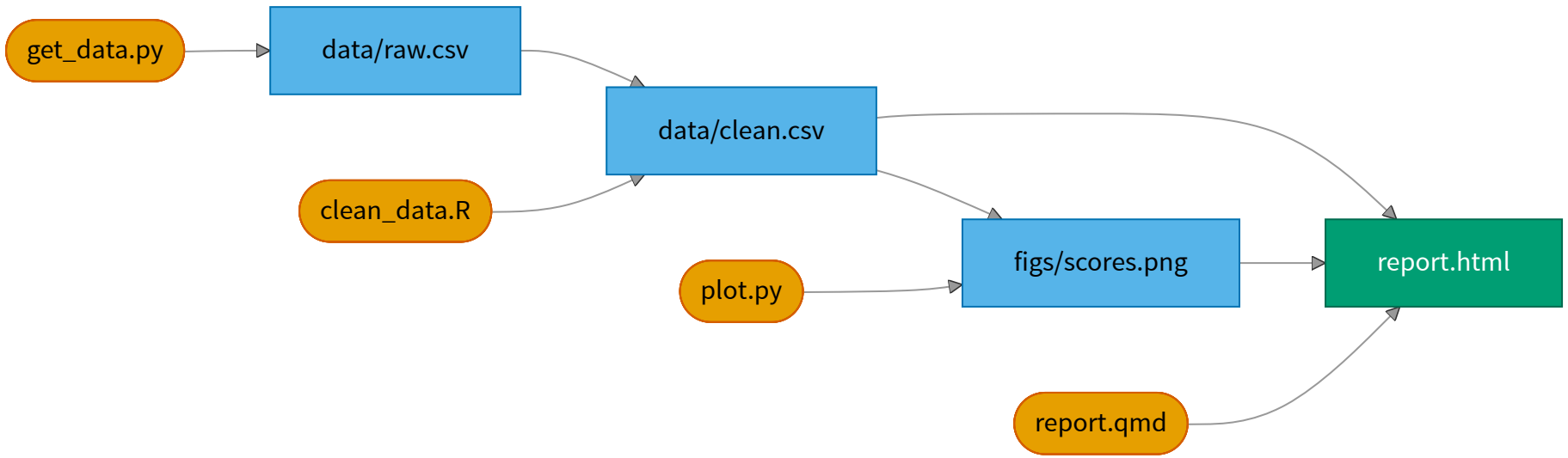
Most data projects are a handful of scripts that read some files and write others, ending in a report. Here is a small one that mixes Python, R, and Quarto.

```
project/
├── get_data.py       downloads the raw data to data/raw.csv
├── clean_data.R      cleans it into data/clean.csv
├── plot.py           draws figs/scores.png from the clean data
└── report.qmd        renders report.html from the clean data and the figure
```

Building the report by hand means running the right commands, in the right order, every time something changes.

```bash
mkdir -p data figs
uv run python get_data.py
Rscript clean_data.R
uv run python plot.py
quarto render report.qmd
```

What depends on what?



If we edit `clean_data.R`, the clean data, the figure, and the report are all out of date but the download is not. Keeping track of this by hand is tedious and error prone, so we would like a tool that does it for us.

make

- A build tool that creates files from other files by following a set of rules you write down.
- Originally created by Stuart Feldman in 1976 at Bell Labs to build C programs, but it knows nothing about any particular language. Anything that reads files and writes files can be automated with make.
- Available on Linux through the system package manager, on macOS with the command line tools, and on Windows through WSL or Rtools.
- The rules usually live in a text file named `Makefile` in the project directory. Run `make --version` to check your installation.

We use GNU make syntax. The main examples also work with GNU make 3.81, the older version supplied with the macOS

Rules

A `Makefile` is a list of rules. Each rule names a target, the prerequisites it is built from, and a recipe of shell commands that builds it.

```
target: prerequisite1 prerequisite2 ...
  recipe line 1
  recipe line 2
  ...
```

- For file rules, paths are relative to make's **working directory**, usually the directory where you run `make`.
- Each recipe line runs in its own shell by default, usually `/bin/sh` on Unix. Your interactive shell settings may not apply.
- In these examples, each recipe line begins with a literal tab character, not spaces.

A first Makefile

One rule per output file, with the script that produces it and the files that script reads as the prerequisites.

```
report.html: report.qmd data/clean.csv figs/scores.png
  quarto render report.qmd

figs/scores.png: plot.py data/clean.csv
  mkdir -p figs
  uv run python plot.py

data/clean.csv: clean_data.R data/raw.csv
  mkdir -p data
  Rscript clean_data.R

data/raw.csv: get_data.py
  mkdir -p data
  uv run python get_data.py
```

The dependency graph determines build order. The first ordinary target is the default; here that is `report.html`. `mkdir -p` creates a directory if needed.

Running make

With no target named, `make` builds the default target and echoes its recipe commands. Each example below starts with no generated files.

```bash
make
```

```
mkdir -p data
uv run python get_data.py
mkdir -p data
Rscript clean_data.R
mkdir -p figs
uv run python plot.py
quarto render report.qmd
```

```bash
make data/clean.csv
```

```
mkdir -p data
uv run python get_data.py
mkdir -p data
Rscript clean_data.R
```

Naming a target builds just that target and whatever it needs.

How make decides what to run

For an ordinary file rule, make rebuilds a missing target or one whose prerequisite has a newer modification time. It updates prerequisites first, then checks the target.

It compares **timestamps**, not file contents.

Running `make` a second time therefore does nothing.

```bash
make
```

```
make: `report.html' is up to date.
```

After the first build we edit `clean_data.R` and run `make` again. Which commands run? What if we had edited `report.qmd` instead, or `get_data.py`?

Predict the rebuild

Start from a completed build. For each change, which outputs need to be rebuilt?

Change	Outputs rebuilt
Edit `clean_data.R`	clean data, figure, report
Edit `report.qmd`	report only
Edit `get_data.py`	raw data, clean data, figure, report
Delete `figs/scores.png`	figure, report
The remote website changes	none: no local prerequisite changed

These predictions assume each successful recipe writes its target. Downloading fresh data needs an explicit refresh

Dry runs

For these rules, `make -n` prints the commands that would run without executing them. **Each column starts from an up-to-date build.**

```bash
touch clean_data.R
make -n
```

```
mkdir -p data
Rscript clean_data.R
mkdir -p figs
uv run python plot.py
quarto render report.qmd
```

```bash
touch report.qmd
make -n
```

```
quarto render report.qmd
```

`touch` changes the modification time without changing contents. A dry run leaves outputs stale; run `make` between these

Tabs, not spaces

The most common error with a new `Makefile` is indenting a recipe with spaces. make only recognizes a tab as the start of a recipe line.

```bash
make
```

```
Makefile:4: *** missing separator.  Stop.
```

Most editors switch to tabs automatically for files named `Makefile`, but if yours converts tabs to spaces you will need to turn that off for this file type.

Recipe lines use separate shells

A `cd` on one recipe line does not change the directory for the next line. Keep related commands on the same line with `&&`.

This does not keep the directory change:

```
preview:
  cd analysis
  Rscript summary.R
```

Run both commands in one shell:

```
preview:
  cd analysis && Rscript summary.R
```

`make -C analysis` changes make's working directory before it reads the `Makefile`. `make -f analysis/Makefile` selects a file but **does not** change directories.

Phony targets

A target can name an action. An `all` target gathers the desired outputs; a `clean` target removes selected generated files.

```
all: report.html

clean:
  rm -f data/clean.csv figs/scores.png report.html
  rm -rf report_files

.PHONY: all clean
```

`.PHONY` makes these names actions even if files named `all` or `clean` exist. `make all` checks its prerequisites; it does not force their recipes to rerun.

This `clean` keeps downloaded raw data. Remove only files the workflow can regenerate.

Variables

Variables keep shared commands in one place. Here `:=` stores a value immediately, and `$(NAME)` inserts it into a recipe.

```
PY := uv run python
R  := Rscript

all: report.html

.PHONY: all

report.html: report.qmd data/clean.csv figs/scores.png
  quarto render report.qmd

figs/scores.png: plot.py data/clean.csv
  mkdir -p figs
  $(PY) plot.py

data/clean.csv: clean_data.R data/raw.csv
  mkdir -p data
  $(R) clean_data.R

data/raw.csv: get_data.py
  mkdir -p data
  $(PY) get_data.py
```

Automatic variables

Inside a recipe make also provides variables that refer to the rule itself, so recipes do not need to repeat file names.

Variable	Meaning
`$@`	the target
`$<`	the first prerequisite
`$^`	all normal prerequisites, without duplicates
`$(@D)`	the directory part of the target

```
data/clean.csv: clean_data.R data/raw.csv
  mkdir -p "$(@D)"
  $(R) "$<"
```

Pattern rules

When several files are built the same way, a `%` in the target and prerequisites acts as a wildcard that must match the same text in both.

```
figs/scores.png: plot_scores.py \
                 data/clean.csv
  mkdir -p "$(@D)"
  $(PY) "$<"

figs/counts.png: plot_counts.py \
                 data/clean.csv
  mkdir -p "$(@D)"
  $(PY) "$<"
```

```
figs/%.png: plot_%.py data/clean.csv
  mkdir -p "$(@D)"
  $(PY) "$<"
```

Since the script is listed first, `$<` is the script for whichever figure is being built. Pattern rules only fire when something asks for a matching target, so the figures still need to be listed as prerequisites of `report.html`.

Wildcards and functions

Make functions use `$(name arguments)`. Here we find source files and derive their output names.

```
QMD  := $(wildcard *.qmd)
HTML := $(patsubst %.qmd,%.html,$(QMD))

all: $(HTML)

.PHONY: all

%.html: %.qmd
  quarto render "$<"
```

- `$(wildcard pattern)` expands to the files matching a shell glob.
- `$(patsubst from,to,text)` rewrites each word in `text` that matches `from`.

This assumes each `.qmd` renders to a same-named HTML file. Add data files, shared configuration, and other inputs as prerequisites when the documents use them.

This course's website

A simplified version of the website's build rules, with repeated paths collected in variables.

```
QMD  := $(wildcard static/slides/*.qmd)
HTML := $(patsubst %.qmd,%.html,$(QMD))
PDF  := $(patsubst %.html,%.pdf,$(HTML))

all: build

build: $(HTML) $(PDF)
  hugo

static/slides/%.html: static/slides/%.qmd
  quarto render "$<"

static/slides/%.pdf: static/slides/%.html
  Rscript -e "renderthis::to_pdf('$<')"

.PHONY: all build
```

`patsubst` changes filename suffixes; `subst` replaces matching text anywhere in a path. The next slide adds the shared

Include shared inputs

The slide rules also need the theme files that Quarto reads. Put shared inputs in a variable, then add them to the prerequisites.

```
THEME := static/slides/slides.scss \
         static/slides/_bootstrap-utilities.scss

static/slides/%.html: static/slides/%.qmd $(THEME)
  quarto render "$<"
```

- Editing the shared theme should rebuild all affected slides.
- Images, included files, and configuration also need dependencies where relevant.
- `:=` expands a variable when it is defined; `=` expands it when used. Either works for fixed command names, but the distinction matters when variables refer to other variables.

Creating directories

If a script expects its output directory to exist, create that directory in the recipe. `mkdir -p` succeeds when the directory already exists.

```
data/raw.csv: get_data.py
  mkdir -p "$(@D)"
  $(PY) get_data.py
```

Alternatively, a directory can be listed as an order-only prerequisite after a `|`. It is created if missing, but its timestamp is ignored, which matters since a directory's time changes whenever a file inside it does.

```
data/raw.csv: get_data.py | data
  $(PY) get_data.py

data:
  mkdir -p "$@"
```

Many outputs: a completion marker

A scraper may produce a changing collection of pages. A completion marker (or sentinel) records that the whole batch finished successfully.

```
data/pages/.done: get_pages.py
  mkdir -p "$(@D)"
  rm -f "$@"
  $(PY) get_pages.py
  touch "$@"

data/listings.csv: parse_pages.R data/pages/.done
  $(R) parse_pages.R
```

Remove the old marker before starting, and create it only after the script succeeds. A failed batch then leaves no marker, so the next build retries it.

What the marker does not track

A marker records completion, not the state of every file in the batch.

- Deleting or editing one saved page does not change `.done`; make will not notice it through this dependency.
- A change on the remote website does not change a local timestamp.
- Refresh explicitly when needed, then rebuild downstream outputs:

```bash
make -B data/pages/.done
make
```

For a fixed set of outputs, GNU make 4.3+ also supports grouped targets (`a.csv b.csv &: script.py`). That syntax is

Environments as prerequisites

`uv run` normally checks and syncs the project environment. A marker can also make environment changes visible to make, so affected outputs are rebuilt.

```
.venv/.synced: pyproject.toml uv.lock
  rm -f "$@"
  uv sync --locked
  touch "$@"

data/raw.csv: get_data.py .venv/.synced
  mkdir -p "$(@D)"
  $(PY) get_data.py
```

`uv sync --locked` fails if the lockfile needs updating. Update it deliberately with `uv lock`, then rebuild. Add this prerequisite to every Python-derived target that should rerun after an environment change.

The marker does not detect manual changes inside `.venv`. R projects can use a similar marker depending on `renv.lock`,

make only sees files

make knows nothing about what a script reads or writes. It only compares the timestamps of the files you list, so the `Makefile` has to describe the project accurately.

- A prerequisite you forget to list means a stale target that make believes is up to date.
- A recipe that does not produce its target means that target is rebuilt on every run.
- Extra outputs are not tracked unless the rules represent them explicitly or use a documented batch strategy.

Prefer explicit output files and list every input that can affect them: data, helper scripts, configuration, and environment lockfiles. A change to a recipe itself does not automatically invalidate its target.

Failed recipes and partial outputs

By default, a nonzero command exit stops that recipe and prevents dependent targets from being built. A script can still leave a partly written output behind.

```
.DELETE_ON_ERROR:

results/fit.json: fit.py data/listings.csv
  mkdir -p "$(@D)"
  $(PY) "$<"
```

`.DELETE_ON_ERROR` removes a target changed by a failed recipe. It does not clean up other files the script wrote, and it only helps when the script reports failure.

For valuable outputs, scripts can write to a temporary file and rename it into place only after success. Completion

Useful options

Option	Effect
`-n`	print recipes instead of running them (see caveat below)
`-B`	rebuild everything, ignoring timestamps
`-j N`	run up to `N` recipes in parallel where the graph allows
`-k`	keep going after an error and build what can still be built
`-s`	do not echo commands
`-f file`	use `file` instead of `Makefile`
`-C dir`	change to `dir` before doing anything

```bash
make -j 4 all
```

With `-j`, independent recipes can run at the same time. Shared files and directories must be represented correctly; do not run `make -j all clean`.

`-n` is not a sandbox: recursive `$(MAKE)` recipes, `+`-prefixed recipes, and commands expanded by `$(shell ...)` can still

Exercise 1

A project has the following pieces,

- `scrape.py` downloads many HTML pages into `data/pages/`
- `parse.R` reads those pages and writes `data/listings.csv`
- `fit.py` reads `data/listings.csv` and writes `results/fit.json`
- `report.qmd` reads both `data/listings.csv` and `results/fit.json` and renders to `report.html`

Write a `Makefile` with `all` and `clean` targets, variables for the Python and R commands, and directory creation where needed. Use a completion marker for the downloaded pages. Let `clean` remove derived outputs while preserving the pages.

After a complete build, predict what reruns if `parse.R` changes, `fit.py` changes, or one downloaded page is deleted. Test the first two with `touch` and `make -n`.

Check your Makefile

Try these checks in the exercise project:

1. Build with no generated files. Every required directory should be created.
2. Run `make` again. No recipe should run.
3. Change one script and inspect `make -n`. Only affected outputs should rebuild.
4. Run `make clean`, then `make -j 4`. The derived outputs should rebuild using the saved pages.
5. Explain why deleting one page can leave the marker looking current. Decide when the project should refresh its download.

Beyond make

make is general purpose and widely available, but it only understands files and timestamps. Workflow tools built for data analysis extend the same idea.

- `targets` for R tracks R objects and declared file targets. It uses hashes of code and data to detect changes and supports parallel execution.
- `Snakemake` uses a Python-based workflow language with explicit inputs and outputs. It adds features for software environments, clusters, and containers.

Both are worth learning when a project outgrows a `Makefile`. For most course projects and papers, make is enough.

Summary

Takeaways

- A `Makefile` describes a project as rules, each with a target file, the prerequisite files it depends on, and the shell commands that build it. Recipe lines start with a tab.
- Make rebuilds missing or outdated targets using modification times. It cannot detect changed web content or unlisted inputs.
- Phony targets like `all` and `clean` give the project a standard interface, and `make -n` shows what would run before anything does.
- Variables, automatic variables, and pattern rules remove repetition, so a single rule can build every figure or every document.
- Make only knows the dependencies you declare. Represent multiple outputs deliberately, handle partial failures, and verify both a fresh build and a second run.

Learning more

- GNU make manual
- minimal make, a short introduction for reproducible research
- Software Carpentry - Automation and Make
- targets and the targets user manual
- Snakemake